

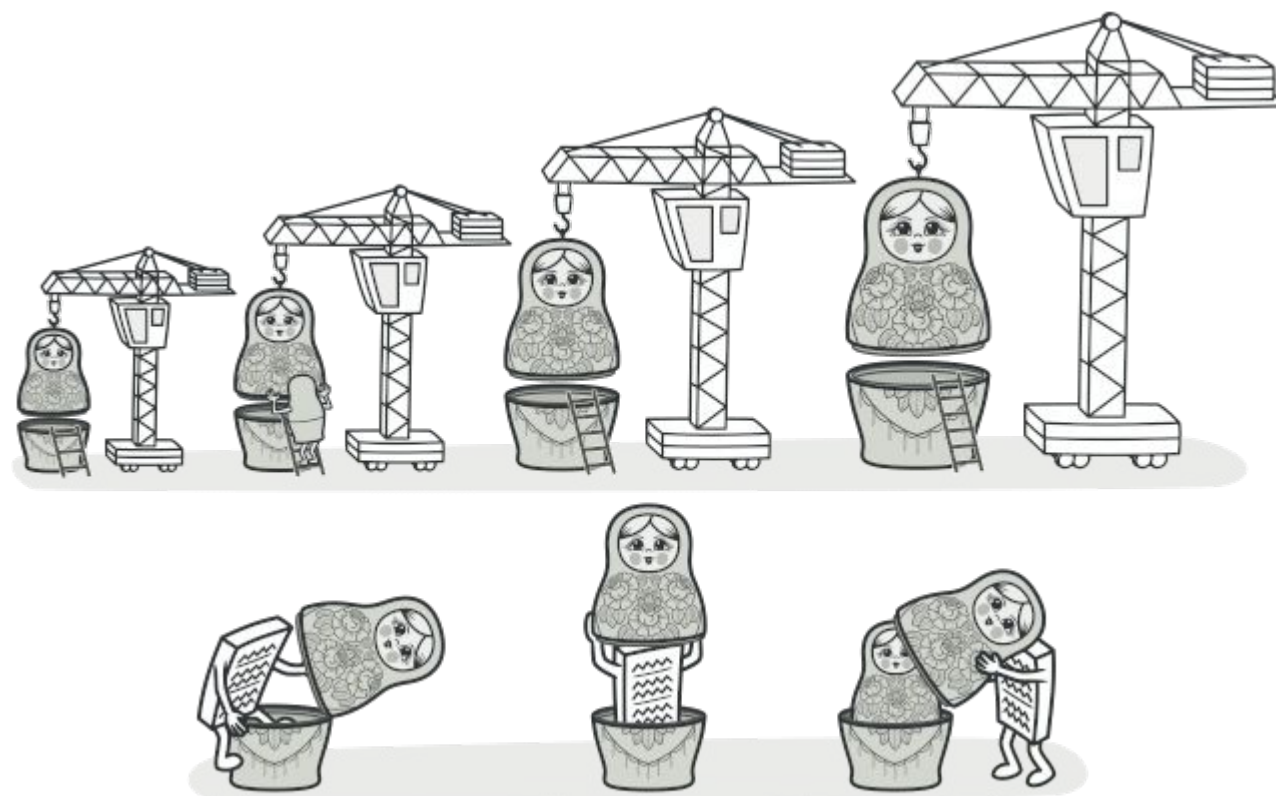


Decorator

*Santiago Cárdenas Jiménez - Stiven
Gómez*

Decorator

Es un patrón de diseño estructural que permite adjuntar nuevos comportamientos a los objetos colocando estos objetos dentro de objetos contenedores especiales que contienen dichos comportamientos.



A la Verdad, por la fe y la ciencia

Decorator

Patrón Estructural

Permite agregar responsabilidades a objetos de forma dinámica, sin modificar su clase original.

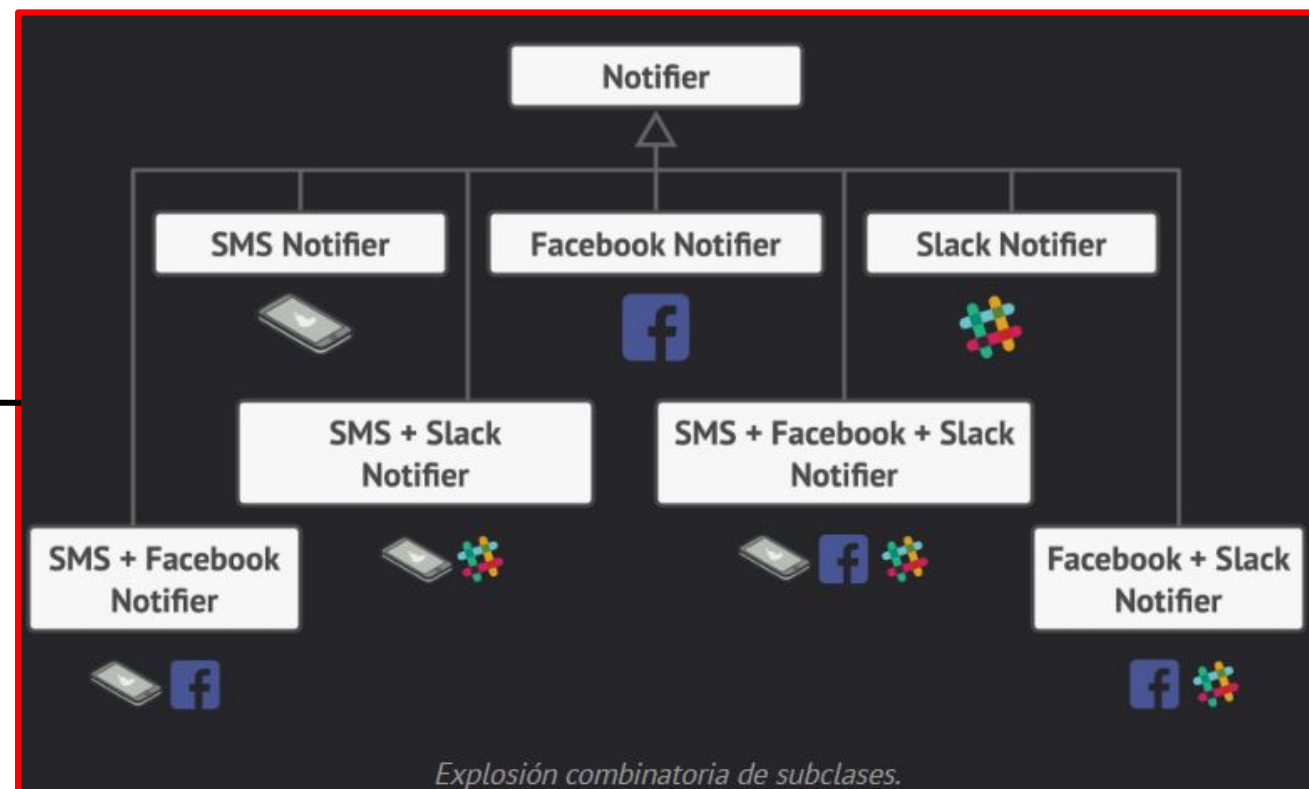
El Concepto de Envoltura

Un "decorador" envuelve al objeto original y añade nuevo comportamiento por encima de él.

Alternativa a la Herencia

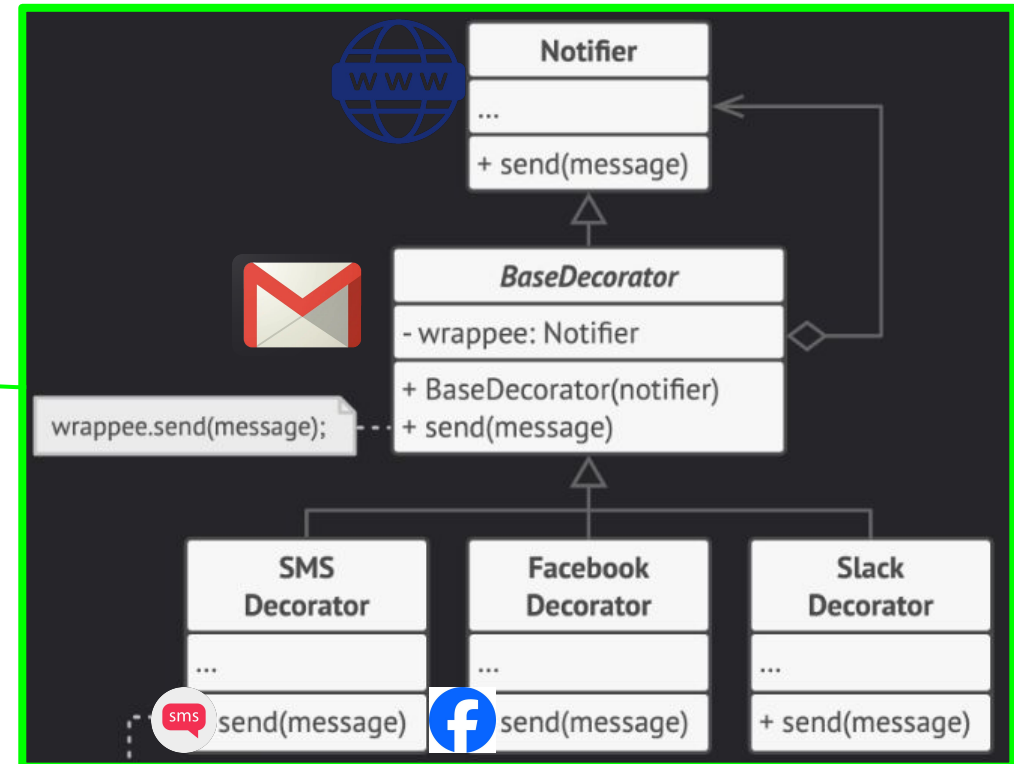
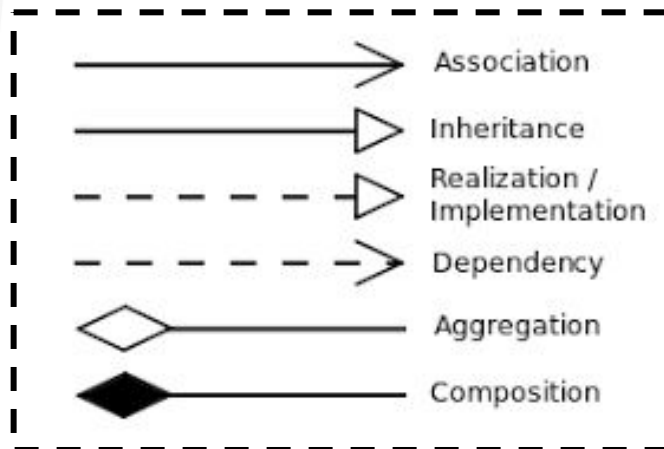
Extiende funcionalidad de manera flexible sin crear jerarquías de clases rígidas.

EXPLOSION DE CLASES



A la Verdad, por la fe y la ciencia

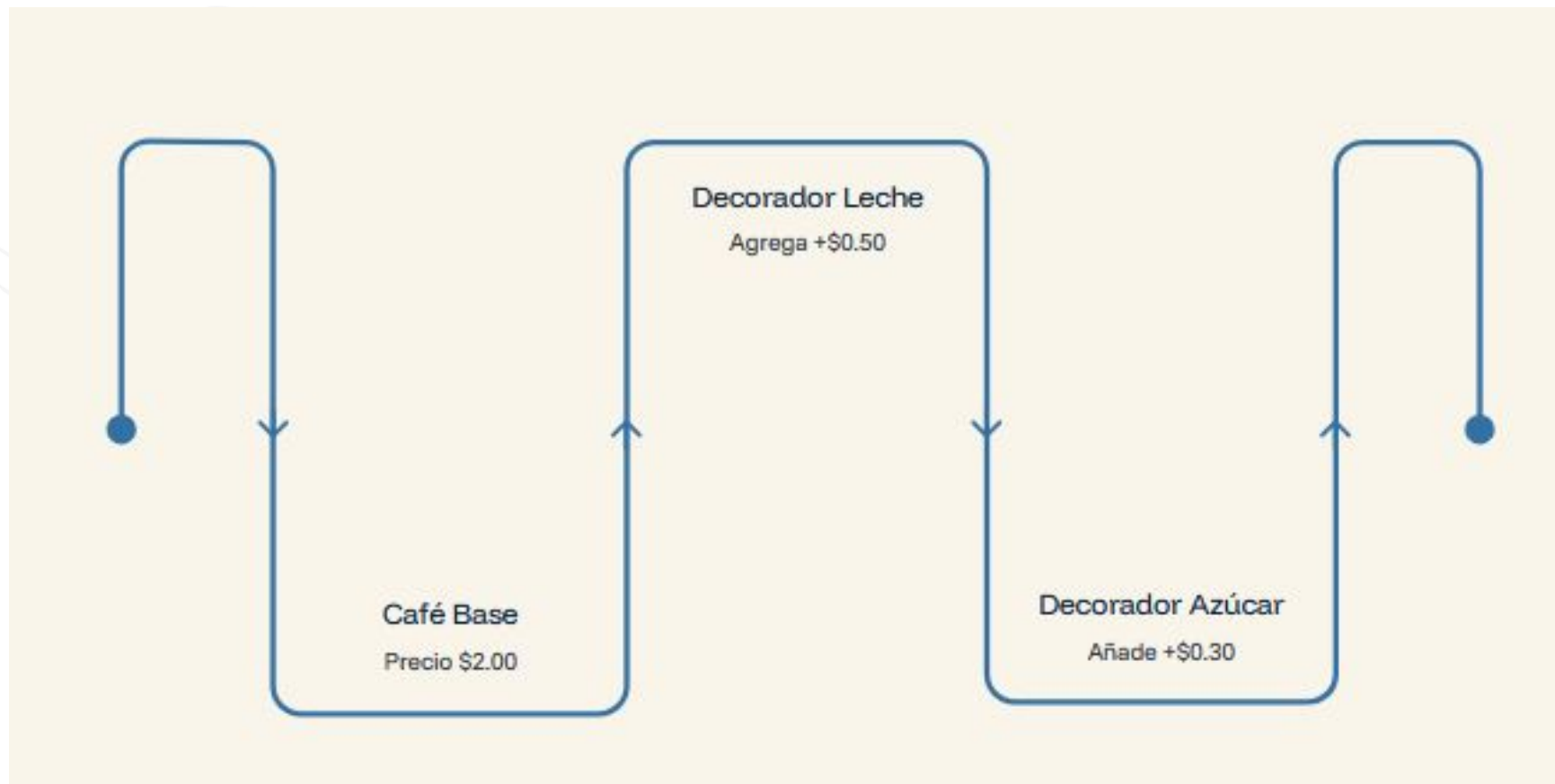
SOLUCIÓN



A la Verdad. por la fe y la ciencia

Composición dinámica

Cada decorador envuelve al anterior, sumando su precio y descripción de forma encadenada. El resultado final es **\$2.80** sin haber creado ninguna clase nueva.



Ventajas del Patrón

Flexibilidad Total

Combina características en tiempo de ejecución sin necesidad de crear nuevas clases para cada combinación.

Responsabilidad Única

Cada decorador hace una sola cosa bien. El código queda limpio, enfocado y fácil de entender.

Composición sobre Herencia

Favorece la composición de objetos, lo que resulta en un diseño más mantenible y menos frágil a largo plazo.

A la Verdad. por la fe y la ciencia

Desventajas a Considerar

El Orden Importa

El orden en que se anidan los decoradores puede cambiar el resultado final de forma inesperada.

Complejidad en Memoria

Cada decorador es un objeto independiente. Cadenas largas generan muchos objetos en memoria.

Debugging Difícil

Rastrear errores en cadenas de decoradores puede ser tedioso y confuso durante el desarrollo.

A la Verdad, por la fe y la ciencia

Bibliografía

- <https://refactoring.guru/design-patterns/decorator>



A la Verdad. por la fe y la ciencia



¡Gracias!